

Cours C++ — POO & Héritage à travers l'exercice Polyset

0. Liste des notions mobilisées dans Polyset

1. Classe abstraite / méthode virtuelle pure
2. Héritage simple
3. Héritage multiple
4. Héritage virtuel (résolution du problème du diamant)
5. Fonctions virtuelles et redéfinition (override implicite en C++98/03)
6. Polymorphisme dynamique / liaison dynamique
7. Destructeur virtuel
8. Forme canonique orthodoxe (Big Four / Orthodox Canonical Form)
9. `const` correctness
10. Composition par référence (vs héritage)
11. Membres référence et suppression des fonctions spéciales (`= delete`)
12. Slicing (découpage d'objet)
13. Downcasting / `static_cast`
14. Encapsulation et visibilité `protected`
15. Surcharge de fonctions (overloading) — à ne pas confondre avec l'override

Chaque notion ci-dessous est utilisée réellement dans le sujet — pas de remplissage générique.

1. Classe abstraite / méthode virtuelle pure

Définition : une classe abstraite contient au moins une méthode déclarée `= 0` (virtuelle pure). Elle ne peut pas être instanciée directement ; elle sert de contrat que les classes filles doivent respecter.

Pourquoi ici : `bag` et `searchable_bag` sont abstraites. `bag` impose l'interface minimale d'un sac (`insert`, `print`, `clear`). `searchable_bag` ajoute le contrat `has()`. Aucune des deux n'est jamais instanciée seule dans `main.cpp` — seuls `searchable_array_bag` et `searchable_tree_bag` (classes concrètes, car elles implémentent *toutes* les méthodes pures héritées) le sont.

Syntaxe

```
cpp
```

```
class bag
{
public:
    virtual void insert(int) = 0;
    virtual void insert(int *, int) = 0;
    virtual void print() const = 0;
    virtual void clear() = 0;
};
```

⚠ Pièges

- Oublier d'implémenter **une seule** des méthodes pures héritées rend la classe fille abstraite *elle aussi*, même sans le vouloir — le compilateur refusera de l'instancier avec une erreur souvent peu claire ("cannot declare variable to be of abstract type").
- `searchable_array_bag` doit implémenter `has()` (venant de `searchable_bag`) mais *aussi* hérite déjà des implémentations concrètes de `insert`, `print`, `clear` via `array_bag` — elle n'a donc **que** `has()` à écrire.

2. Héritage simple

Définition : une classe fille hérite des membres d'une seule classe mère directe.

Pourquoi ici : `array_bag : virtual public bag` et `tree_bag : virtual public bag` sont des héritages simples — chacune n'a qu'un seul parent direct (`bag`), même si le mot-clé `virtual` (voir section 4) est présent.

Syntaxe

```
cpp

class array_bag : virtual public bag { /* ... */ };
```

⚠ Pièges

- Le mode d'héritage (`public`/`protected`/`private`) change la visibilité des membres hérités dans la classe fille. Ici tout est `public` — logique, car on veut que l'interface polymorphe (`bag*`, `searchable_bag*`) reste utilisable depuis l'extérieur.

3. Héritage multiple

Définition : une classe hérite de **plusieurs** classes directement.

Pourquoi ici : c'est le cœur de l'exercice.

```
cpp
```

```
class searchable_array_bag : public array_bag, public searchable_bag
```

`searchable_array_bag` combine **deux** héritages : la structure de données (`array_bag`), qui fournit `data`, `size`, et les implémentations concrètes de `insert`/`print`/`clear`) et le contrat de recherche (`searchable_bag`, qui exige `has()`). Idem pour `searchable_tree_bag` avec `tree_bag`.

Syntaxe

```
cpp

class searchable_tree_bag : public tree_bag, public searchable_bag
{
    public:
        bool has(int) const;
        /* ... */
};
```

⚠ Pièges

- L'héritage multiple crée un risque de **duplication** si plusieurs chemins mènent à la même classe ancêtre (ici : `array_bag` → `bag` ET `searchable_bag` → `bag`, donc `bag` est atteint deux fois). Sans précaution, l'objet contiendrait **deux sous-objets** `bag` distincts et ambigus. → voir section 4.

4. Héritage virtuel (problème du diamant)

Définition : quand une classe est héritée deux fois via deux chemins différents, le mot-clé `virtual` devant l'héritage force le compilateur à ne créer **qu'une seule instance partagée** de la classe ancêtre commune, au lieu d'une par chemin.

Pourquoi ici : c'est exactement la situation du "diamant" :



Sans `virtual`, `searchable_array_bag` contiendrait deux sous-objets `bag` distincts, et un appel comme `sab.insert(5)` (via un pointeur `bag*`) serait **ambigu** pour le compilateur : lequel des deux `bag` faut-il utiliser ? C'est pour cette raison que `array_bag` et `searchable_bag` héritent **toutes deux virtuellement** de `bag` :

```
cpp
```

```
class array_bag : virtual public bag { /* ... */ };
class searchable_bag : virtual public bag { /* ... */ };
```

Résultat : `searchable_array_bag` n'a qu'un seul sous-objet `bag`, partagé par les deux chemins d'héritage.

⚠ Pièges

- L'héritage virtuel doit être déclaré **au niveau des classes intermédiaires** (`array_bag`, `searchable_bag`), pas au niveau de `searchable_array_bag`. C'est trop tard pour le faire une fois qu'on est au dernier niveau.
- **Ordre d'initialisation particulier** : dans une hiérarchie avec héritage virtuel, c'est la classe la **plus dérivée** (`searchable_array_bag`) qui est responsable d'appeler explicitement le constructeur de la base virtuelle (`bag`) — les classes intermédiaires n'ont plus ce rôle. Ici ce n'est pas visible car `bag` n'a pas de constructeur défini explicitement (classe purement abstraite sans données), mais c'est un piège classique dans d'autres exercices similaires.
- Coût : l'héritage virtuel ajoute une indirection (pointeur vers la table virtuelle de base) — négligeable ici, mais bon à savoir pour la théorie.

5. Fonctions virtuelles et redéfinition

Définition : une méthode (`virtual`) peut être redéfinie (override) dans une classe fille ; l'appel effectif se décide à **l'exécution** selon le type réel de l'objet pointé, pas selon le type statique du pointeur/référence.

Pourquoi ici : `insert`, `print`, `clear` sont virtuelles dans `bag`, et `has` est virtuelle dans `searchable_bag`. Elles sont redéfinies dans `array_bag`/`tree_bag` (pour les trois premières) et dans `searchable_array_bag`/`searchable_tree_bag` (pour `has`).

Syntaxe

```
cpp

class searchable_array_bag : public array_bag, public searchable_bag
{
    public:
        bool has(int) const;    // redéfinit searchable_bag::has (virtuelle pure)
};
```

⚠ Pièges

- La signature doit être **identique** (type de retour covariant accepté, mais paramètres et `const` doivent matcher) sinon ce n'est plus une redéfinition mais une **surcharge accidentelle qui masque** la méthode virtuelle parente (piège très fréquent : oublier le

`const` fait échouer l'override silencieusement en C++98 sans `override` keyword pour le détecter).

- Ici pas de mot-clé `override` (C++11) utilisé dans le sujet — donc aucune vérification automatique du compilateur ; il faut être rigoureux à la main sur les signatures.
-

6. Polymorphisme dynamique / liaison dynamique

Définition : appeler une méthode virtuelle via un pointeur ou une référence vers la classe de base déclenche, à l'exécution, l'implémentation de la classe **réelle** de l'objet (et non celle du type déclaré).

Pourquoi ici : `main.cpp` manipule les objets **uniquement** via `searchable_bag*` :

cpp

```
searchable_bag *t = new searchable_tree_bag;
searchable_bag *a = new searchable_array_bag;
t->insert(value);    // appelle tree_bag::insert malgré le type statique searcha
a->has(value);       // appelle searchable_array_bag::has
```

C'est la **raison d'être** de toute la hiérarchie : pouvoir traiter un `array_bag` et un `tree_bag` de façon uniforme via une interface commune.

⚠ Pièges

- Le polymorphisme **ne fonctionne qu'avec pointeurs ou références**, jamais par valeur (voir slicing, section 12).
 - Si une méthode n'est **pas** déclarée `virtual` dans la classe de base, l'appel via un pointeur de base utilisera toujours l'implémentation de base (liaison **statique**), même si l'objet réel est dérivé — erreur classique.
-

7. Destructeur virtuel

Définition : le destructeur de la classe de base doit être `virtual` dès qu'on prévoit de détruire un objet dérivé via un pointeur vers la base (`delete basePtr;`), sinon seul le destructeur de la classe de base est appelé → fuite mémoire / destruction incomplète.

Pourquoi ici : dans `main.cpp`, on fait `new searchable_tree_bag` stocké dans un `searchable_bag*`. Si un jour ce pointeur est détruit par `delete`, il **faudrait** que la chaîne de destructeurs (`~searchable_tree_bag → ~tree_bag → ~searchable_bag → ~bag`) soit respectée.

⚠ Pièges

- Dans le code fourni, `bag` **n'a pas de destructeur virtuel explicite** ! C'est un point sensible : bien que `main.cpp` ne fasse jamais `delete t;` / `delete a;` explicitement (fuite mémoire volontaire ou oubli de l'exercice), en toute rigueur d'examen, **toute**

classe destinée au polymorphisme doit déclarer un destructeur virtuel dans sa classe de base. C'est une bonne pratique à mentionner même si le sujet fourni ne le fait pas.

- Règle mnémotechnique : "si une classe a au moins une méthode virtuelle, elle doit avoir un destructeur virtuel."

8. Forme canonique orthodoxe (Orthodox Canonical Form)

Définition : ensemble de 4 fonctions spéciales qu'une classe C++ "propre" doit définir explicitement : constructeur par défaut, constructeur de copie, `operator=`, destructeur.

Pourquoi ici : exigence explicite du sujet ("All classes should be under orthodox canonical form"). `array_bag`, `tree_bag`, `searchable_array_bag`, `searchable_tree_bag` implémentent les quatre. `set` y déroge volontairement (voir section 10-11).

Syntaxe (exemple `searchable_array_bag`)

```
cpp
searchable_array_bag(); // par défaut
searchable_array_bag(const searchable_array_bag& source); // copie
searchable_array_bag& operator=(const searchable_array_bag& source); // affectat
~searchable_array_bag(); // destruct
```

La copie **délègue** à la classe parente :

```
cpp
searchable_array_bag::searchable_array_bag(const searchable_array_bag& source)
: array_bag(source) {}
```

⚠ Pièges

- Dans une hiérarchie, le constructeur de copie et `operator=` de la classe fille doivent **explicitement appeler/déléguer** à ceux du parent, sinon le parent est copié avec son constructeur par défaut (perte de données) : `array_bag::operator=(source)` doit être appelé à la main dans `searchable_array_bag::operator=` — c'est fait correctement dans la solution.
- `array_bag` gère de la mémoire brute (`int *data`) → sans constructeur de copie/`operator=` bien écrits, on aurait une **copie superficielle** (shallow copy) menant à un double `delete` ou des pointeurs pendants (dangling pointers).

9. `const` correctness

Définition : marquer `const` les méthodes qui ne modifient pas l'état de l'objet, et les paramètres/references qui ne doivent pas être modifiés.

Pourquoi ici : `has()` et `print()` sont `const` partout (`bool has(int) const`, `void print() const`) — logique, consulter/afficher un sac ne le modifie pas. Le sujet insiste explicitement : *"Don't forget the const."*

Syntaxe

```
cpp

bool searchable_array_bag::has(int value) const
{
    for (int i = 0; i < this->size; i++)
        if (this->data[i] == value)
            return (true);
    return (false);
}
```

Dans `main.cpp` on voit aussi : `const searchable_array_bag tmp(...)` puis `tmp.print(); tmp.has(1);` — cela **ne compilerait pas** si `print()`/`has()` n'étaient pas `const`.

⚠ Pièges

- Une méthode virtuelle redéfinie doit matcher la constance de la version de base — `const` fait partie de la signature. Oublier `const` dans une classe fille = ce n'est plus un override, c'est une méthode différente qui masque la version virtuelle (bug silencieux, voir section 5).
- `insert()` n'est volontairement **pas** `const` (elle modifie l'état) — savoir distinguer quelles méthodes doivent l'être ou non fait partie de l'examen.

10. Composition par référence (vs héritage)

Définition : au lieu d'hériter d'une classe, on peut **posséder une référence/pointeur** vers un objet d'une autre classe et déléguer les appels ("has-a" plutôt que "is-a").

Pourquoi ici : `set` **n'hérite de rien**. Elle contient `searchable_bag& bag` et délègue :

```
cpp

bool set::has(int value) const { return (bag.has(value)); }
void set::insert(int value) { if (!(this->has(value))) bag.insert(value); }
```

C'est un choix de conception délibéré : un `set` n'est pas un sac au sens strict (il ne doit jamais accepter de doublons), donc hériter aurait été sémantiquement faux — on préfère la **composition**, qui "enveloppe" et transforme le comportement d'un `searchable_bag` existant.

⚠ Pièges

- `set` ne possède **pas** le bag, elle le **référence** : détruire le `set` ne détruit pas le bag sous-jacent (pas de `delete` dans `~set()`). Il faut bien distinguer "avoir une référence vers" et "posséder/gérer le cycle de vie de".
-

11. Membres référence et `= delete`

Définition : un membre déclaré comme référence (`T&`) doit être initialisé **une seule fois**, à la construction, et ne peut jamais être réassigné à un autre objet ensuite. Cela a des conséquences en cascade sur les fonctions spéciales.

Pourquoi ici :

```
cpp

class set
{
private:
    searchable_bag& bag;
public:
    set() = delete;
    set(const set& source) = delete;
    set& operator=(const set& source) = delete;
    set(searchable_bag& s_bag);
    /* ... */
};
```

- Pas de constructeur par défaut possible : une référence **doit** être initialisée dès la construction, on ne peut pas la laisser "vide". D'où `set() = delete`.
- Pas d'`operator=` possible : réaffecter `bag` à une autre référence après coup est **interdit par le langage** (une référence est liée pour toujours à son objet initial). D'où `set& operator=(...) = delete`.
- Le constructeur de copie est supprimé par cohérence de conception (éviter que deux `set` se disputent le même bag sous-jacent de façon ambiguë).

⚠ Pièges

- C'est l'**exact inverse** de la forme canonique demandée pour les autres classes (section 8) — c'est volontaire et doit être justifié à l'examen : "parce que `set` contient une référence, pas parce qu'on a oublié".
 - Sans `= delete` explicite, le compilateur **essaierait de générer automatiquement** un constructeur par défaut et un `operator=`, et échouerait à la compilation avec des messages d'erreur peu clairs à cause du membre référence. Le `= delete` rend l'intention explicite et l'erreur claire dès l'appel.
-

12. Slicing (découpage d'objet)

Définition : quand un objet dérivé est copié (par valeur) dans une variable ou un paramètre du **type de base**, seule la partie "base" est copiée — la partie spécifique à la classe dérivée est "tranchée" (perdue). Le polymorphisme cesse alors de fonctionner sur cette copie.

Pourquoi ici : c'est un risque latent avec `array_bag`/`tree_bag` si on les manipulait par valeur au lieu de par référence/pointeur. Dans `main.cpp`, ce risque est **évit**é en manipulant toujours des pointeurs (`searchable_bag *t`, `*a`). La ligne suivante illustre en revanche une copie **intentionnelle et sûre** car on reste dans le même type concret :

```
cpp
const searchable_array_bag tmp(static_cast<searchable_array_bag &>(*a));
```

Ici on copie bien un `searchable_array_bag` complet vers un `searchable_array_bag` — pas de slicing car aucun changement de type de base au passage.

⚠ Pièges

- Passer un objet **par valeur** à une fonction attendant `bag` (au lieu de `bag&` ou `bag*`) provoquerait un slicing — d'ailleurs impossible ici car `bag` est abstraite (on ne peut pas la copier "à plat" comme objet complet), ce qui **protège partiellement** contre l'erreur — mais le principe reste fondamental à comprendre pour l'examen.

13. Downcasting / `static_cast`

Définition : convertir un pointeur/référence de classe de base vers une classe dérivée plus spécifique. Contrairement à l'upcasting (implicite et toujours sûr), le downcasting nécessite une conversion explicite et n'est sûr **que si on est certain** du type réel de l'objet.

Pourquoi ici :

```
cpp
const searchable_array_bag tmp(static_cast<searchable_array_bag &>(*a));
```

`a` est déclaré `searchable_bag *`, mais on **sait** (car on l'a construit ainsi juste avant) que l'objet pointé est réellement un `searchable_array_bag`. On force donc le compilateur à traiter `*a` comme tel.

⚠ Pièges

- `static_cast` **ne vérifie rien à l'exécution** — si le type réel de l'objet n'était pas `searchable_array_bag`, ce serait un comportement indéfini (accès mémoire invalide). C'est différent de `dynamic_cast`, qui vérifie à l'exécution (mais nécessite la RTTI et lève/renvoie un échec détectable). Ici `static_cast` est acceptable **uniquement** parce que le programmeur maîtrise le contexte (`a`) vient d'être créé

comme `(new searchable_array_bag)`).

14. Encapsulation et visibilité `protected`

Définition : les membres `protected` sont invisibles depuis l'extérieur de la classe, mais accessibles depuis les classes **dérivées**.

Pourquoi ici :

```
cpp

class array_bag : virtual public bag
{
    protected:
        int *data;
        int size;
        /* ... */
};
```

`searchable_array_bag::has()` accède directement à `this->data` et `this->size` — possible uniquement parce qu'ils sont `protected` dans `array_bag` (et non `private`). Même logique pour `tree_bag::node *tree` (`protected`), utilisé par `searchable_tree_bag::search()`.

⚠ Pièges

- Si `data`/`size`/`tree` avaient été `private`, `searchable_array_bag`/`searchable_tree_bag` n'auraient **pas pu** y accéder directement malgré l'héritage — il aurait fallu passer par des accesseurs publics/protégés. C'est un piège de conception classique : bien choisir `protected` vs `private` selon si les classes filles ont besoin d'un accès direct.

15. Surcharge de fonctions (overloading) — à distinguer de l'override

Définition : plusieurs fonctions de même nom mais de signatures (paramètres) différentes dans la **même** classe. À ne pas confondre avec la redéfinition (override) qui se fait dans une classe **fil**le avec la **même** signature.

Pourquoi ici : `bag` déclare **deux** versions virtuelles pures de `insert` :

```
cpp

virtual void insert(int) = 0;
virtual void insert(int *, int) = 0;
```

Ce sont deux méthodes distinctes (surcharge), toutes deux virtuelles, toutes deux à redéfinir séparément dans les classes concrètes. `set::insert(int)` et

`set::insert(int*, int)` suivent le même principe — et `insert(int*, int)` appelle en interne `insert(int)` élément par élément pour respecter l'unicité.

⚠ Pièges

- Une classe fille qui ne redéfinit **qu'une seule** des deux surcharges laisse l'autre comme virtuelle pure non implémentée → la classe reste abstraite par erreur (piège fréquent, à relier à la section 1).

16. Table récapitulative

Notion	Où elle apparaîtrait dans Polyset	Piège principal à retenir
Classe abstraite / virtuelle pure	<code>bag</code> , <code>searchable_bag</code>	Oublier UNE méthode pure rend la classe fille abstraite malgré elle
Héritage simple	<code>array_bag</code> , <code>tree_bag</code> héritent de <code>bag</code>	Le mode d'héritage (<code>public</code>) fixe la visibilité héritée
Héritage multiple	<code>searchable_array_bag : array_bag, searchable_bag</code>	Risque de duplication d'ancêtre commun (<code>bag</code>)
Héritage virtuel	<code>virtual public bag</code> partout	Doit être déclaré aux niveaux intermédiaires, pas à la fin
Fonctions virtuelles / redéfinition	<code>insert</code> , <code>print</code> , <code>clear</code> , <code>has</code>	Signature (dont <code>const</code>) doit matcher exactement, sinon masquage silencieux
Polymorphisme dynamique	<code>searchable_bag *t/a</code> dans <code>main.cpp</code>	Ne fonctionne que via pointeur/référence, jamais par valeur
Destructeur virtuel	absent explicitement dans <code>bag</code>	Base polymorphe sans destructeur virtuel = destruction incomplète via <code>delete basePtr</code>
Forme canonique orthodoxe	<code>array_bag</code> , <code>tree_bag</code> , <code>searchable_*_bag</code>	La copie/affectation fille doit déléguer explicitement au parent
<code>const</code> correctness	<code>has() const</code> , <code>print() const</code>	<code>const</code> fait partie de la signature virtuelle — l'oublier casse l'override
Composition par référence	<code>set</code> contient <code>searchable_bag&</code>	Le <code>set</code> ne possède pas le bag, il ne le

Notion	Où elle apparaît dans Polyset	Piège principal à retenir
		détruit jamais
Membre référence + <code>= delete</code>	<code>set() = delete</code> , copie/affectation supprimées	Référence = liaison figée à vie, donc pas de constructeur par défaut ni d' <code>operator=</code> possibles
Slicing	évitée via pointeurs dans <code>main.cpp</code>	Copier un dérivé dans une variable de type base tranche les données spécifiques
Downcasting / <code>static_cast</code>	<code>static_cast<searchable_array_bag*>(*a)</code>	Aucune vérification à l'exécution — sûr seulement si le type réel est connu
Encapsulation <code>protected</code>	<code>data</code> / <code>size</code> dans <code>array_bag</code> , <code>tree</code> dans <code>tree_bag</code>	<code>private</code> aurait bloqué l'accès direct depuis les classes filles
Surcharge (overloading)	<code>insert(int)</code> / <code>insert(int*, int)</code>	Ne pas confondre avec l'override ; chaque surcharge virtuelle pure doit être redéfinie séparément

17. Comment les notions s'enchaînent (le fil rouge de l'exercice)

1. **Classe abstraite** (`bag`) définit un contrat minimal.
2. **Héritage simple** donne deux implémentations concrètes différentes (`array_bag`, `tree_bag`).
3. Une **deuxième classe abstraite** (`searchable_bag`) ajoute un contrat orthogonal (chercher).
4. **L'héritage multiple** combine les deux (`searchable_array_bag` = structure + recherche), ce qui crée mécaniquement un **diamant**, résolu par **l'héritage virtuel**.
5. Le tout n'a de sens que grâce au **polymorphisme dynamique** : `main.cpp` traite `array_bag` et `tree_bag` de façon interchangeable via `searchable_bag*`, ce qui **exige** que toutes les méthodes utilisées soient `virtual` et correctement redéfinies

(**const correctness** incluse).

6. Un **destructeur virtuel** aurait dû accompagner ce polymorphisme (point faible du sujet fourni, à corriger en théorie).
7. Enfin, **(set)** montre l'alternative à l'héritage : la **composition par référence**, qui impose ses propres règles (**membre référence, (= delete)**) — un contraste pédagogique volontaire avec la **forme canonique** rigoureuse exigée partout ailleurs.

Comprendre cet enchaînement (abstraction → héritage multiple → diamant → héritage virtuel → polymorphisme → alternative par composition) est la clé pour répondre à des questions de cours qui combinent plusieurs notions en une seule question d'examen.